



沐曦通用 GPU 与 MXMACA[®] 软件栈

MXMACA[®] 编译器内建函数编程指南

OG-26013-000-F5_V01

2026-08-03

声明

版权所有 ©2026 沐曦集成电路（上海）股份有限公司。保留所有权利。

本文档中呈现的信息属于沐曦集成电路（上海）股份有限公司和/或其附属公司（以下统称为“沐曦股份”），非经沐曦股份事先书面许可，任何实体或个人均不得获得本文档的副本，且无权以任何方式处理本文档，包括但不限于使用、复制、修改、合并、出版、发行、销售或传播本文档的部分或全部。

本文档内容仅供参考，不提供任何形式的、明示或暗示的保证，包括但不限于对适销性、适用于任何目的和/或不侵权的保证。在任何情况下，沐曦股份均不对因本文档引起的、由本文档造成的、或与之相关的任何索赔、损害或其他责任负责。

沐曦股份保留自行决定随时更改、修改、添加或删除本文档的部分或全部的权利。沐曦股份保留最终解释权。

沐曦、MetaX 和其他沐曦图标是沐曦股份的商标。本文档中提及的所有其他商标和商品名称均为其各自所有者的财产。

目录

1 概述	1
2 类型定义	2
3 接口介绍	3
3.1 访存函数	3
3.1.1 __builtin_mxc_ldg_*_predicator	3
3.1.2 __builtin_mxc_stg_*_predicator	5
3.1.3 __builtin_mxc_ldg_*_predicator_ret	7
3.1.4 __builtin_mxc_ldg_*_bsm	9
3.1.5 __builtin_mxc_load_shared_trans_*	10
3.1.6 __builtin_mxc_arrive	10
3.1.7 __builtin_mxc_arrive_gymcnt	11
3.1.8 __builtin_mxc_arrive_bsmcnt	12
3.1.9 __builtin_mxc_barrier_ex	12
3.1.10 __builtin_mxc_barrier_and_wait*	13
3.2 比较函数	15
3.2.1 __builtin_mxc_uicmp*	15
3.2.2 __builtin_mxc_sicmp*	16
3.2.3 __builtin_mxc_fcmp*	16
3.3 类型转换函数	17
3.3.1 __builtin_mxc_cvt_f32totf32	17
3.3.2 __builtin_mxc_cvt_pk_f32tou8	18
3.3.3 __builtin_mxc_b*_cast_to_f32	18
3.4 混洗函数	19
3.4.1 __builtin_mxc_mov_shfl	19
3.4.2 __builtin_mxc_update_shfl	21
3.4.3 __builtin_mxc_bsm_bpermute	22
3.4.4 __builtin_mxc_bsm_permute	23
3.4.5 __builtin_mxc_byte_perm	24
3.5 浮点乘加函数	25
3.5.1 __builtin_mxc_pk_fma_f32	25
3.5.2 __builtin_mxc_pk_fma_bf16	25
3.5.3 __builtin_mxc_pk_fma_f16	26
3.5.4 __builtin_mxc_fma_bf16	26
3.6 矩阵乘加函数	26
3.6.1 __builtin_mxc_mma_16x16x16f16	26
3.6.2 __builtin_mxc_mma_16x16x4f32	27
3.6.3 __builtin_mxc_mma_16x16x16i8	28
3.6.4 __builtin_mxc_mma_16x16x32i8	29

3.6.5	__builtin_mxc_mma_16x16x16bf16	29
3.6.6	__builtin_mxc_mma_16x16x4f64	30
3.6.7	__builtin_mxc_mma_16x16x8tf32	31
3.7	常用计算函数	31
3.7.1	__builtin_mxc_mad_wide_i32/u32	31
3.7.2	__builtin_mxc_pk_mul_f32	32
3.7.3	__builtin_mxc_pk_add_f32	32
3.7.4	__builtin_mxc_ubfe	33
3.7.5	__builtin_mxc_sbfe	34
3.7.6	__builtin_mxc_alignbit	34
3.7.7	__builtin_mxc_cos*	35
3.7.8	__builtin_mxc_sin*	35
3.7.9	__builtin_mxc_rcp*	36
3.7.10	__builtin_mxc_sqrt*	36
3.7.11	__builtin_mxc_rsq*	36
3.7.12	__builtin_mxc_log_clampf	37
3.8	其他	37
3.8.1	__builtin_mxc_get_time	37
3.8.2	__builtin_mxc_read_xmsk	38
3.8.3	__builtin_mxc_readfirstlane	38
3.8.4	__builtin_mxc_readlane	38
3.8.5	__builtin_mxc_writelane	39
3.8.6	__builtin_mxc_nop	40

表目录

表 3.1	__builtin_mxc_ldg_*_predicator 参数说明	4
表 3.2	__builtin_mxc_stg_*_predicator 参数说明	6
表 3.4	__builtin_mxc_ldg_*_bsm 参数说明	9
表 3.5	__builtin_mxc_arrive 参数说明	11
表 3.6	__builtin_mxc_arrive_gvmcnt 参数说明	11
表 3.7	__builtin_mxc_arrive_bsmcnt 参数说明	12
表 3.8	__builtin_mxc_barrier_ex 参数说明	13
表 3.9	__builtin_mxc_barrier_and_wait 参数说明	14
表 3.10	__builtin_mxc_uicmp 参数说明	15
表 3.11	__builtin_mxc_fcmp 参数说明	16
表 3.12	__builtin_mxc_cvt_pk_f32tou8 参数说明	18
表 3.13	__builtin_mxc_mov_shfl 参数说明	19
表 3.14	__builtin_mxc_update_shfl 参数说明	21
表 3.15	__builtin_mxc_bsm_bpermute 参数说明	23
表 3.16	__builtin_mxc_bsm_permute 参数说明	23
表 3.17	__builtin_mxc_byte_perm 参数说明	24
表 3.18	__builtin_mxc_mma_16x16x16f16 参数说明	27
表 3.19	__builtin_mxc_mma_16x16x4f32 参数说明	27
表 3.20	__builtin_mxc_mma_16x16x16i8 参数说明	28
表 3.21	__builtin_mxc_mma_16x16x32i8 参数说明	29
表 3.22	__builtin_mxc_mma_16x16x16bf16 参数说明	30
表 3.23	__builtin_mxc_mma_16x16x4f64 参数说明	30
表 3.24	__builtin_mxc_mma_16x16x8tf32 参数说明	31
表 3.25	__builtin_mxc_mad_wide 参数说明	32
表 3.26	__builtin_mxc_ubfe 参数说明	33
表 3.27	__builtin_mxc_sbfe 参数说明	34
表 3.28	__builtin_mxc_alignbit 参数说明	35
表 3.29	__builtin_mxc_readlane 参数说明	39
表 3.30	__builtin_mxc_writelane 参数说明	39
表 3.31	__builtin_mxc_nop 参数说明	40

更新记录

版本	日期	更新说明
V01	2026-08-03	首次发布

1 概述

本文档介绍了 MXCC 编译器支持的一系列内建函数及其功能，用于指导用户利用这些接口来编写更高效的 MACA C++ 代码。

2 类型定义

文档中使用到的一些 vector 类型定义如下：

```
typedef __NATIVE_VECTOR__(1, unsigned char) v1u8;

typedef __NATIVE_VECTOR__(4, char) v4i8;
typedef __NATIVE_VECTOR__(8, char) v8i8;
typedef __NATIVE_VECTOR__(16, char) v16i8;

typedef __NATIVE_VECTOR__(1, unsigned short) v1u16;

typedef __NATIVE_VECTOR__(2, short) v2i16;
typedef __NATIVE_VECTOR__(4, short) v4i16;
typedef __NATIVE_VECTOR__(8, short) v8i16;

typedef __NATIVE_VECTOR__(1, unsigned int) v1u32;
typedef __NATIVE_VECTOR__(2, unsigned int) v2u32;
typedef __NATIVE_VECTOR__(4, unsigned int) v4u32;

typedef __NATIVE_VECTOR__(2, int) v2i32;
typedef __NATIVE_VECTOR__(4, int) v4i32;

typedef __NATIVE_VECTOR__(2, _Float16) v2f16;
typedef __NATIVE_VECTOR__(4, _Float16) v4f16;
typedef __NATIVE_VECTOR__(8, _Float16) v8f16;

typedef __NATIVE_VECTOR__(2, float) v2f32;
typedef __NATIVE_VECTOR__(4, float) v4f32;

typedef __NATIVE_VECTOR__(4, double) v4f64;
```


3 接口介绍

3.1 访存函数

3.1.1 __builtin_mxc_ldg*_predicator

功能描述

从全局内存中加载数据。相比普通加载操作，该操作的活跃线程掩码 exec_mask 来自于函数参数中两个数值的比较结果。

接口声明

```
u1u8 __builtin_mxc_ldg_b8_predicator(void *global_addr, int offset, bool ret0_en,
↪ bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
↪ cmp_operand2, int cmp_flag);
v1u16 __builtin_mxc_ldg_b16_predicator(void *global_addr, int offset, bool ret0_en,
↪ bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
↪ cmp_operand2, int cmp_flag);
v1u32 __builtin_mxc_ldg_b32_predicator(void *global_addr, int offset, bool ret0_en,
↪ bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
↪ cmp_operand2, int cmp_flag);
v2i16 __builtin_mxc_ldg_b32_predicator_v2s(void *global_addr, int offset, bool
↪ ret0_en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪ unsigned cmp_operand2, int cmp_flag);
v2i16 __builtin_mxc_ldg_b32_predicator_v2h(void *global_addr, int offset, bool
↪ ret0_en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪ unsigned cmp_operand2, int cmp_flag);
v4i8 __builtin_mxc_ldg_b32_predicator_v4c(void *global_addr, int offset, bool ret0_
↪ en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪ unsigned cmp_operand2, int cmp_flag);
v2u32 __builtin_mxc_ldg_b64_predicator(void *global_addr, int offset, bool ret0_en,
↪ bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
↪ cmp_operand2, int cmp_flag);
v4i16 __builtin_mxc_ldg_b64_predicator_v4s(void *global_addr, int offset, bool
↪ ret0_en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪ unsigned cmp_operand2, int cmp_flag);
v4f16 __builtin_mxc_ldg_b64_predicator_v4h(void *global_addr, int offset, bool
↪ ret0_en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪ unsigned cmp_operand2, int cmp_flag);
v8i8 __builtin_mxc_ldg_b64_predicator_v8c(void *global_addr, int offset, bool ret0_
↪ en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪ unsigned cmp_operand2, int cmp_flag);
v4u32 __builtin_mxc_ldg_b128_predicator(void *global_addr, int offset, bool ret0_
↪ en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
```

(下页继续)

(续上页)

```

↪ unsigned cmp_operand2, int cmp_flag);
v8i16 __builtin_mxc_ldg_b128_predicate_v8s(void *global_addr, int offset, bool
↪ ret0_en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪ unsigned cmp_operand2, int cmp_flag);
v8f16 __builtin_mxc_ldg_b128_predicate_v8h(void *global_addr, int offset, bool
↪ ret0_en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪ unsigned cmp_operand2, int cmp_flag);
v16i8 __builtin_mxc_ldg_b128_predicate_v16c(void *global_addr, int offset, bool
↪ ret0_en, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪ unsigned cmp_operand2, int cmp_flag);

```

参数

表 3.1 __builtin_mxc_ldg_*_predicate 参数说明

名称	描述
return_value	从内容中载入的值
global_addr	全局内存地址
offset	全局内存偏移
ret0_en	当参数为 true 时，非活跃线程返回 0
saddr_flag	暗示编译器使用 streg(single thread register) 存储全局内存地址
pred_neg	当参数为 true 时，将活跃线程掩码取反
is_async	当参数为 true 时，用户需要主动插入 arrive 操作进行访存同步，而非依赖编译器处理。请谨慎使用
cmp_opnd1	用于生成活跃线程掩码的第一个比较操作数
cmp_opnd2	用于生成活跃线程掩码的第二个比较操作数
cmp_flag	可用的比较操作。取值范围及含义如下： MACA_ICMP_EQ: exec_mask = (cmp_operand1 == cmp_operand2) MACA_ICMP_NE: exec_mask = (cmp_operand1 != cmp_operand2) MACA_ICMP_UGT: exec_mask = (cmp_operand1 > cmp_operand2), unsigned MACA_ICMP_UGE: exec_mask = (cmp_operand1 >= cmp_operand2), unsigned MACA_ICMP_ULT: exec_mask = (cmp_operand1 < cmp_operand2) unsigned MACA_ICMP_ULE: exec_mask = (cmp_operand1 <= cmp_operand2), unsigned MACA_ICMP_SGT: exec_mask = (cmp_operand1 > cmp_operand2), signed MACA_ICMP_SGE: exec_mask = (cmp_operand1 >= cmp_operand2), signed MACA_ICMP_SLT: exec_mask = (cmp_operand1 < cmp_operand2), signed MACA_ICMP_SLE: exec_mask = (cmp_operand1 <= cmp_operand2), signed

代码示例

```

using v1u32 = __NATIVE_VECTOR__(1, unsigned int);
__device__ v1u32 test_builtin_mxc_ldg_b32_predicate(int *ptr, int a) {

```

(下页继续)

(续上页)

```

    vlu32 val = __builtin_mxc_ldg_b32_predicator(ptr, 0, false, false, false, false,
    ↪threadIdx.x, a, MACA_ICMP_ULT);
    return val;
}

```

3.1.2 __builtin_mxc_stg_*_predicator

功能描述

将数据写入全局内存。相比普通写入操作，该操作的活跃线程掩码 exec_mask 来自于函数参数中两个数值的比较结果。

接口声明

```

void __builtin_mxc_stg_b8_predicator(int *global_addr, int offset, vlu8 data, bool
    ↪saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned cmp_
    ↪operand2, int cmp_flag);
void __builtin_mxc_stg_b16_predicator(int *global_addr, int offset, vlu16 data,
    ↪bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b32_predicator(int *global_addr, int offset, vlu32 data,
    ↪bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b32_predicator_v2s(int *global_addr, int offset, v2i16 data,
    ↪ bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b32_predicator_v2h(int *global_addr, int offset, v2f16 data,
    ↪ bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b32_predicator_v4c(int *global_addr, int offset, v4i8 data,
    ↪bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b64_predicator(int *global_addr, int offset, v2u32 data,
    ↪bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b64_predicator_v4s(int *global_addr, int offset, v4i16 data,
    ↪ bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b64_predicator_v4h(int *global_addr, int offset, v4f16 data,
    ↪ bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b64_predicator_v8c(int *global_addr, int offset, v8i8 data,
    ↪bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b128_predicator(int *global_addr, int offset, v4u32 data,
    ↪bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1, unsigned
    ↪cmp_operand2, int cmp_flag);

```

(下页继续)

(续上页)

```

void __builtin_mxc_stg_b128_predicator_v8s(int *global_addr, int offset, v8i16
↪data, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪unsigned cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b128_predicator_v8h(int *global_addr, int offset, v8f16
↪data, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪unsigned cmp_operand2, int cmp_flag);
void __builtin_mxc_stg_b128_predicator_v16c(int *global_addr, int offset, v16i8
↪data, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
↪unsigned cmp_operand2, int cmp_flag);

```

参数

表 3.2 __builtin_mxc_stg_*_predicator 参数说明

名称	描述
global_addr	全局内存地址
offset	全局内存偏移
data	需要被写入全局内存的数据
saddr_flag	暗示编译器使用 streg(single thread register) 存储全局内存地址
pred_neg	当参数为 true 时，将活跃线程掩码取反
is_async	当参数为 true 时，用户需要主动插入 arrive 操作进行访存同步，而非依赖编译器处理。请谨慎使用
cmp_operand1	用于生成活跃线程掩码的第一个比较操作数
cmp_operand2	用于生成活跃线程掩码的第二个比较操作数
cmp_flag	可用的比较操作。取值范围及含义如下： MACA_ICMP_EQ: exec_mask = (cmp_operand1 == cmp_operand2) MACA_ICMP_NE: exec_mask = (cmp_operand1 != cmp_operand2) MACA_ICMP_UGT: exec_mask = (cmp_operand1 > cmp_operand2), unsigned MACA_ICMP_UGE: exec_mask = (cmp_operand1 >= cmp_operand2), unsigned MACA_ICMP_ULT: exec_mask = (cmp_operand1 < cmp_operand2) unsigned MACA_ICMP_ULE: exec_mask = (cmp_operand1 <= cmp_operand2), unsigned MACA_ICMP_SGT: exec_mask = (cmp_operand1 > cmp_operand2), signed MACA_ICMP_SGE: exec_mask = (cmp_operand1 >= cmp_operand2), signed MACA_ICMP_SLT: exec_mask = (cmp_operand1 < cmp_operand2), signed MACA_ICMP_SLE: exec_mask = (cmp_operand1 <= cmp_operand2), signed

代码示例

```

__global__ void test_builtin_mxc_stg_b32_predicator(void *ptr, unsigned val, int
↪a) {
    __builtin_mxc_stg_b32_predicator(ptr, 0, val, false, false, false, threadIdx.x,
↪a, MACA_ICMP_ULT);
}

```

3.1.3 __builtin_mxc_ldg*_predicator_ret

功能描述

从全局内存中加载数据。相比普通加载操作，该操作的活跃线程掩码来自于函数参数中两个数值的比较结果。相比 __builtin_mxc_ldg*_predicator，该操作可以指定非活跃线程的返回值。

接口声明

```
vlu8 __builtin_mxc_ldg_b8_predicator_ret(void *global_addr, int offset, vlu8 ret_val,
    ↪ bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
    ↪ unsigned cmp_operand2, int cmp_flag);
vlu16 __builtin_mxc_ldg_b16_predicator_ret(void *global_addr, int offset, vlu16
    ↪ ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
    ↪ unsigned cmp_operand2, int cmp_flag);
vlu32 __builtin_mxc_ldg_b32_predicator_ret(void *global_addr, int offset, vlu32
    ↪ ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
    ↪ unsigned cmp_operand2, int cmp_flag);
v2i16 __builtin_mxc_ldg_b32_predicator_v2s_ret(void *global_addr, int offset,
    ↪ v2i16 ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_
    ↪ operand1, unsigned cmp_operand2, int cmp_flag);
v2f16 __builtin_mxc_ldg_b32_predicator_v2h_ret(void *global_addr, int offset,
    ↪ v2f16 ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_
    ↪ operand1, unsigned cmp_operand2, int cmp_flag);
v4i8 __builtin_mxc_ldg_b32_predicator_v4c_ret(void *global_addr, int offset, v4i8
    ↪ ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
    ↪ unsigned cmp_operand2, int cmp_flag);
v2u32 __builtin_mxc_ldg_b64_predicator_ret(void *global_addr, int offset, v2u32
    ↪ ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
    ↪ unsigned cmp_operand2, int cmp_flag);
v4i16 __builtin_mxc_ldg_b64_predicator_v4s_ret(void *global_addr, int offset,
    ↪ v4i16 ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_
    ↪ operand1, unsigned cmp_operand2, int cmp_flag);
v4f16 __builtin_mxc_ldg_b64_predicator_v4h_ret(void *global_addr, int offset,
    ↪ v4f16 ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_
    ↪ operand1, unsigned cmp_operand2, int cmp_flag);
v8i8 __builtin_mxc_ldg_b64_predicator_v8c_ret(void *global_addr, int offset, v8i8
    ↪ ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
    ↪ unsigned cmp_operand2, int cmp_flag);
v4u32 __builtin_mxc_ldg_b128_predicator_ret(void *global_addr, int offset, v4u32
    ↪ ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_operand1,
    ↪ unsigned cmp_operand2, int cmp_flag);
v8i16 __builtin_mxc_ldg_b128_predicator_v8s_ret(void *global_addr, int offset,
    ↪ v8i16 ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_
    ↪ operand1, unsigned cmp_operand2, int cmp_flag);
v8f16 __builtin_mxc_ldg_b128_predicator_v8h_ret(void *global_addr, int offset,
    ↪ v8f16 ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_
    ↪ operand1, unsigned cmp_operand2, int cmp_flag);
v16i8 __builtin_mxc_ldg_b128_predicator_v16c_ret(void *global_addr, int offset,
```

(下页继续)

(续上页)

```
↪v16i8 ret_val, bool saddr_flag, bool pred_neg, bool is_async, unsigned cmp_
↪operand1, unsigned cmp_operand2, int cmp_flag);
```

参数

名称	描述
return_value	从内容中载入的值
global_addr	全局内存地址
offset	全局内存偏移
ret_val	非活跃线程的返回值
saddr_flag	暗示编译器使用 streg(single thread register) 存储全局内存地址
pred_neg	当参数为 true 时，将活跃线程掩码取反
is_async	当参数为 true 时，用户需要主动插入 arrive 操作进行访存同步，而非依赖编译器处理。请谨慎使用
cmp_operand1	用于生成活跃线程掩码的第一个比较操作数
cmp_operand2	用于生成活跃线程掩码的第二个比较操作数
cmp_flag	可用的比较操作。取值范围及含义如下： MACA_ICMP_EQ: exec_mask = (cmp_operand1 == cmp_operand2) MACA_ICMP_NE: exec_mask = (cmp_operand1 != cmp_operand2) MACA_ICMP_UGT: exec_mask = (cmp_operand1 > cmp_operand2), unsigned MACA_ICMP_UGE: exec_mask = (cmp_operand1 >= cmp_operand2), unsigned MACA_ICMP_ULT: exec_mask = (cmp_operand1 < cmp_operand2) unsigned MACA_ICMP_ULE: exec_mask = (cmp_operand1 <= cmp_operand2), unsigned MACA_ICMP_SGT: exec_mask = (cmp_operand1 > cmp_operand2), signed MACA_ICMP_SGE: exec_mask = (cmp_operand1 >= cmp_operand2), signed MACA_ICMP_SLT: exec_mask = (cmp_operand1 < cmp_operand2), signed MACA_ICMP_SLE: exec_mask = (cmp_operand1 <= cmp_operand2), signed

代码示例

```
__device__ v1u32 test_builtin_mxc_ldg_b32_predicator_ret(int *ptr, v1u32 ret_val,
↪int a) {
    v1u32 val = __builtin_mxc_ldg_b32_predicator_ret(ptr, 0, ret_val, false, false,
↪false, threadIdx.x, a, MACA_ICMP_ULT);
    return val;
}
```

3.1.4 __builtin_mxc_ldg_*_bsm

功能描述

从全局内存中加载数据并写入共享内存。

接口声明

```
v1u32 __builtin_mxc_ldg_b32_bsm(__shared__ void *shared_addr, void *global_addr,
↪int offset, size_t mask, bool ret0_en, bool saddr_flag, bool pred_neg, bool is_
↪async);
v2u32 __builtin_mxc_ldg_b64_bsm(__shared__ void *shared_addr, void *global_addr,
↪int offset, size_t mask, bool ret0_en, bool saddr_flag, bool pred_neg, bool is_
↪async);
v4u32 __builtin_mxc_ldg_b128_bsm(__shared__ void *shared_addr, void *global_addr,
↪int offset, size_t mask, bool ret0_en, bool saddr_flag, bool pred_neg, bool is_
↪async);
```

参数

表 3.4 __builtin_mxc_ldg_*_bsm 参数说明

名称	描述
return_value	用于传递给特定同步函数 __builtin_mxc_barrier_and_wait* 的标志位，非实际的加载数据。如果使用 __syncthreads() 来保证该函数执行完成，则返回值可省略
shared_addr	写入数据的共享内存地址
global_addr	加载数据的全局内存地址
offset	共享内存地址和全局内存地址共用的常量偏移
mask	只能填-1。当前无实际意义
ret0_en	当参数为 true 时，非活跃线程返回 0
saddr_flag	暗示编译器使用 streg(single thread register) 存储全局内存地址
pred_neg	当参数为 true 时，将活跃线程掩码取反
is_async	当参数为 true 时，用户需要主动插入 arrive 操作进行访存同步，而非依赖编译器处理。请谨慎使用

代码示例

```
__device__ void test_builtin_mxc_ldg_b32_bsm(int *ptr) {
    extern __shared__ int shared_ptr[];
    v1u32 ret_flag = __builtin_mxc_ldg_b32_bsm(shared_ptr, ptr, 0, -1, false, false,
↪false, false);
    __builtin_mxc_barrier_and_wait1(1, ret_flag);
}
// or
```

(下页继续)

(续上页)

```
__device__ void test_builtin_mxc_ldg_b32_bsm(int *ptr) {
    extern __shared__ int shared_ptr[];
    __builtin_mxc_ldg_b32_bsm(shared_ptr, ptr, 0, -1, false, false, false, false);
    __syncthreads();
    // then load data from shared_ptr
}
```

3.1.5 __builtin_mxc_load_shared_trans_*

架构支持限制

offload arch 需要大于等于 xcore1500。例如 -offload-arch=xcore1500、-offload-arch=xcore1520 等。

功能描述

从共享内存中加载特定 shape 的矩阵并进行转置操作。

接口声明

```
int64_t __builtin_mxc_load_shared_trans_4x16(__shared__ int64_t *shared_ptr);
int64_t __builtin_mxc_load_shared_trans_4x16_i64(__shared__ int64_t *shared_ptr);
v4i16 __builtin_mxc_load_shared_trans_4x16_v4i16(__shared__ v4i16 *shared_ptr);
int64_t __builtin_mxc_load_shared_trans_8x16(__shared__ int64_t *shared_ptr);
int64_t __builtin_mxc_load_shared_trans_8x16_i64(__shared__ int64_t *shared_ptr);
v8i8 __builtin_mxc_load_shared_trans_8x16_v8i8(__shared__ v8i8 *shared_ptr);
```

代码示例

```
__device__ int64_t test_load_shared_trans(__shared__ int64_t *shared_ptr) {
    int64_t val = __builtin_mxc_load_shared_trans_4x16(shared_ptr);
    return val;
}
```

3.1.6 __builtin_mxc_arrive

功能描述

内存栅栏，支持全局内存和共享内存的处理。当访存操作队列中未完成的操作数量达到或低于目标值时，代码才可以继续向后执行。

接口声明

```
void __builtin_mxc_arrive(unsigned flag);
```

参数

表 3.5 __builtin_mxc_arrive 参数说明

名称	描述
flag	flag 中不同 bit 的含义如下： flag[5:0] = gvm_cnt, 全局内存队列剩余未完成数量的目标值 flag[6] = gvm_cnt_enable, 为 1 时使能全局内存栅栏 flag[10:7] = bsm_cnt, 共享内存队列剩余未完成数量的目标值 flag[12] = bsm_cnt_enable, 为 1 时使能共享内存栅栏 当 flag 为 0 时, 表示必须等待前面所有全局内存和共享内存操作执行完成。

代码示例

```
__global__ void test_builtin_mxc_arrive_global(int *ptr, int *data) {
    int val = ptr[0];
    __builtin_mxc_arrive(64); // (1 << 6)
    data[0] = val;
}

__global__ void test_builtin_mxc_arrive_shared(int *data) {
    extern __shared__ int shared_ptr[];
    int shared_val = shared_ptr[threadIdx.x];
    __builtin_mxc_arrive(4096); // (1 << 12);
    data[0] = shared_val;
}
```

3.1.7 __builtin_mxc_arrive_gvmcnt

功能描述

全局内存栅栏。当全局内存访问操作队列中未完成的操作数量达到或低于目标值时, 代码才可以继续向后执行。

接口声明

```
void __builtin_mxc_arrive_gvmcnt(unsigned gvmcnt);
```

参数

表 3.6 __builtin_mxc_arrive_gvmcnt 参数说明

名称	描述
gvmcnt	全局内存队列中剩余未完成数量的目标值。有效值范围: 0~63

代码示例

```
__global__ void test_builtin_mxc_arrive_gvmcnt(int *ptr, int *data) {
    int val = ptr[0];
    __builtin_mxc_arrive_gvmcnt(0);
```

(下页继续)

(续上页)

```
data[0] = val;  
}
```

3.1.8 __builtin_mxc_arrive_bsmcnt

功能描述

共享内存栅栏。当共享内存访问操作队列中未完成的操作数量达到或低于目标值时，代码才可以继续向后执行。

接口声明

```
void __builtin_mxc_arrive_bsmcnt(unsigned bsmcnt);
```

参数

表 3.7 __builtin_mxc_arrive_bsmcnt 参数说明

名称	描述
bsmcnt	全局内存队列中剩余未完成数量的目标值。有效值范围：0~15

代码示例

```
__global__ void test_builtin_mxc_arrive_bsmcnt(int *data) {  
    __shared__ int ptr[];  
    int val = ptr[0];  
    __builtin_mxc_arrive_bsmcnt(0);  
    data[0] = val;  
}
```

3.1.9 __builtin_mxc_barrier_ex

功能描述

指令屏障。block 内所有线程到执行到该位置后才可以继续向后执行。

接口声明

```
void __builtin_mxc_barrier_ex(int flag);
```

参数

表 3.8 __builtin_mxc_barrier_ex 参数说明

参数	描述
flag	当值为 0 时，产生指令屏障和内存栅栏。编译器会根据代码中内存访问操作类型自动添加相应的内存栅栏。 当值为 1 时，产生指令屏障和共享内存栅栏。编译器只有当代码中存在共享内存访问操作时才会自动添加共享内存栅栏。 当值为 2 时，仅产生指令屏障。编译器不会自动插入任何的内存栅栏。 当值为 3 时，不会产生指令屏障和内存栅栏。此时该函数仅用于阻止一些编译优化行为，不会产生任何的同步操作。

代码示例

```
__global__ void test_swap_warp_data(int *data) {
    __shared__ int sharedData[128];
    int id = threadIdx.x + blockIdx.x * blockDim.x;
    int warpId = id / 64;
    int laneId = threadIdx.x % 64;

    sharedData[id] = data[id];

    // Swap data by shared memory. So barrier_shared is enough.
    __builtin_mxc_barrier_ex(1);

    int targetWarpId = warpId ^ 1;
    int targetId = targetWarpId * 64 + laneId;

    int val = sharedData[targetId];
    data[id] = val;
}
```

3.1.10 __builtin_mxc_barrier_and_wait*

功能描述

内存栅栏。该函数只与 __builtin_mxc_ldg*_bsm 配合使用，可以实现针对全局内存到共享内存加载操作的局部等待，从而在代码中构建更加高效灵活的数据访问与计算流水线。

接口声明

```
void __builtin_mxc_barrier_and_wait1(int scope, v1u32 ret_flag); // Used for 32bit
↪load
void __builtin_mxc_barrier_and_wait2(int scope, v2u32 ret_flag); // Used for 64bit
↪load
void __builtin_mxc_barrier_and_wait4(int scope, v4u32 ret_flag); // Used for
↪128bit load
```

参数

表 3.9 __builtin_mxc_barrier_and_wait 参数说明

名称	描述
scop	值为 0 时，仅有内存栅栏语义。当共享内存数据不存在跨 warp 共享时使用 值为 1 时，同时具备内存栅栏和指令屏障语义，功能与 __syncthreads() 相同。 当共享内存数据存在跨 warp 共享时使用
ret_flag	配对的 __builtin_mxc_ldg_*_bsm 的返回值，用来建立两个函数间的 use-def 链

代码示例

```

__device__ void test_builtin_mxc_ldg_b32_bsm(int *ptr) {
    extern __shared__ int shared_ptr[];
    v1u32 ret_flag = __builtin_mxc_ldg_b32_bsm(shared_ptr, ptr, 0, -1, false, false,
↪false, false);
    __builtin_mxc_barrier_and_wait1(1, ret_flag);
}

__device__ void test_builtin_mxc_ldg_b64_bsm(long *ptr) {
    extern __shared__ long shared_ptr[];
    v2u32 ret_flag = __builtin_mxc_ldg_b64_bsm(shared_ptr, ptr, 0, -1, false, false,
↪false, false);
    __builtin_mxc_barrier_and_wait2(0, ret_flag);
}

// Pseudo code to build software memory and compute pipeline.
__global__ void test1(..) {
    ...
    __builtin_mxc_ldg_b64_bsm(shared_ptr1, ptr1, 0, -1, false, false, false,
↪false);
    __builtin_mxc_ldg_b64_bsm(shared_ptr2, ptr2, 0, -1, false, false, false,
↪false);
    __syncthreads(); // wait all memcopy from global to shared at the same time.
    compute();
    ...
}

__global__ void test2(..) {
    ...
    v2u32 ret_flag1 = __builtin_mxc_ldg_b64_bsm(shared_ptr1, ptr1, 0, -1, false,
↪false, false, false);
    v2u32 ret_flag2 = __builtin_mxc_ldg_b64_bsm(shared_ptr2, ptr2, 0, -1, false,
↪false, false, false);
    __builtin_mxc_barrier_and_wait2(0, ret_flag1); // only wait first memcopy
    compute1();
    __builtin_mxc_barrier_and_wait2(0, ret_flag2); // wait second memcopy
    compute2();
    ...
}

```

(下页继续)

(续上页)

}

3.2 比较函数

3.2.1 __builtin_mxc_uicmp*

功能描述

32 或 64 位无符号整数比较运算。函数产生一个 64bit 结果，每个 bit 表示当前 warp 内相应线程的比较结果。

接口声明

```
uint64_t __builtin_mxc_uicmp(unsigned int src0, unsigned int src1, int ICMP_OP);
uint64_t __builtin_mxc_uicmpl(uint64_t src0, uint64_t src1, int ICMP_OP);
```

参数

表 3.10 __builtin_mxc_uicmp 参数说明

名称	描述
src0	左操作数
src1	右操作数
ICMP_OP	运算符枚举值。取值范围如下： MACA_ICMP_EQ：等于 MACA_ICMP_NE：不等于 MACA_ICMP_UGT：无符号大于 MACA_ICMP_UGE：无符号大于等于 MACA_ICMP_ULT：无符号小于 MACA_ICMP_ULE：无符号小于等于 MACA_ICMP_SGT：有符号大于 MACA_ICMP_SGE：有符号大于等于 MACA_ICMP_SLT：有符号小于 MACA_ICMP_SLE：有符号小于等于

代码示例

```
__global__ void test_mxc_uicmp(uint64_t& res, uint64_t& lres, unsigned int a,
↪ unsigned int b, uint64_t x, uint64_t y) {

    res = __builtin_mxc_uicmp(a, b, MACA_ICMP_EQ);

    lres = __builtin_mxc_uicmpl(x, y, MACA_ICMP_NE);
}
```

3.2.2 __builtin_mxc_sicmp*

功能描述

32 或 64 位有符号整数比较运算。函数产生一个 64bit 结果，每个 bit 表示当前 warp 内相应线程的比较结果。

接口声明

```
uint64_t __builtin_mxc_sicmp(int src0, int src1, int ICMP_OP);
uint64_t __builtin_mxc_sicmpl(long src0, long src1, int ICMP_OP);
```

参数

参考 __builtin_mxc_uicmp*

代码示例

```
__global__ void test_mxc_sicmp(uint64_t& res, uint64_t& lres, int a, int b, long x,
    ↪ long y) {
    res = __builtin_mxc_sicmp(a, b, MACA_ICMP_EQ);
    lres = __builtin_mxc_sicmpl(x, y, MACA_ICMP_NE);
}
```

3.2.3 __builtin_mxc_fcmp*

功能描述

浮点比较运算。函数产生一个 64bit 结果，每个 bit 表示当前 warp 内相应线程的比较结果。

接口声明

```
uint64_t __builtin_mxc_fcmp(double src0, double src1, int fcmp_op);
uint64_t __builtin_mxc_fcmpf(float src0, float src1, int fcmp_op);
uint64_t __builtin_mxc_fcmph(_Float16 src0, _Float16 src1, int fcmp_op);
```

参数

表 3.11 __builtin_mxc_fcmp 参数说明

名称	描述
src0	左操作数
src1	右操作数

下页继续

表 3.11 – 续上页

名称	描述
ICMP_OP	运算符枚举值。取值范围如下： CMP_EQ：等于 CMP_LT：小于 CMP_LE：小于等于 CMP_GT：大于 CMP_GE：大于等于 CMP_NEQ：不等于 CMP_O：有序比较，当两个操作数均不是 NaN 时返回 1 CMP_U：无序比较，当两个操作数任意一个为 NaN 时返回 1

代码示例

```
__global__ void test_mxc_fcmp(uint64_t& dres, uint64_t& fres, uint64_t& hres,
                             double da, double db, float fa, float fb, _Float16
→ha, _Float16 hb) {
    dres= __builtin_mxc_fcmp(da, db, 1);
    fres = __builtin_mxc_fcmpf(fa, fb, 2);
    hres = __builtin_mxc_fcmph(ha, hb, 3);
}
```

3.3 类型转换函数

3.3.1 __builtin_mxc_cvt_f32totf32

架构支持限制

offload arch 需要大于等于 xcore1500。例如-offload-arch=xcore1500、-offload-arch=xcore1520 等。

功能描述

将 float 类型数据转换为 tf32 类型。

接口声明

```
float __builtin_mxc_cvt_f32totf32(float);
```

代码示例

```
__global__ void test_cvt_f32totf32(float *in, float *out) {
    int tid = threadIdx.x;
    out[tid] = __builtin_mxc_cvt_f32totf32(in[tid]);
}
```

3.3.2 __builtin_mxc_cvt_pk_f32tou8

功能描述

将一个 float 类型的数据转换为 8bit 无符号整数，并将该 8bit 无符号整数替换到特定数据的指定字节位置。

接口声明

```
unsigned int __builtin_mxc_cvt_pk_f32tou8(float src0, unsigned int src1, unsigned
↪int src2);
```

参数

表 3.12 __builtin_mxc_cvt_pk_f32tou8 参数说明

名称	描述
return_value	返回值
src0	待转换的浮点数据
src1	转换后数据插入位置的索引
src2	被替换局部数据的值

代码示例

```
__global__ void test(int *out, float a, float b, float c, float d) {
    out[0] = __builtin_mxc_cvt_pk_f32tou8(a, 0, out[0]);
    out[0] = __builtin_mxc_cvt_pk_f32tou8(b, 1, out[0]);
    out[0] = __builtin_mxc_cvt_pk_f32tou8(c, 2, out[0]);
    out[0] = __builtin_mxc_cvt_pk_f32tou8(d, 3, out[0]);
}
```

3.3.3 __builtin_mxc_b*_cast_to_f32

功能描述

将输入数据的某个字节以无符号 8bit 整数数值格式转换为 float 类型。

接口声明

```
float __builtin_mxc_b0_cast_to_f32(int S0); // convert S0.u[7:0] to float
float __builtin_mxc_b1_cast_to_f32(int S0); // convert S0.u[15:8] to float
float __builtin_mxc_b2_cast_to_f32(int S0); // convert S0.u[23:16] to float
float __builtin_mxc_b3_cast_to_f32(int S0); // convert S0.u[31:24] to float
```

代码示例


```
__global__ void test_byte_to_float(int* in, float* out){
    out[0] = __builtin_mxc_b0_cast_to_f32(in[0]);
    out[1] = __builtin_mxc_b1_cast_to_f32(in[0]);
    out[2] = __builtin_mxc_b2_cast_to_f32(in[0]);
    out[3] = __builtin_mxc_b3_cast_to_f32(in[0]);
}
```

3.4 混洗函数

3.4.1 __builtin_mxc_mov_shfl

功能描述

一个 warp 中 64 个线程，每 16 个连续线程叫做一个 row，分别为 row0 (thread 0~15)、row1 (thread 16~31)、row2 (thread 32~47) 和 row3 (thread 48~63)。__builtin_mxc_mov_shfl_ 函数分别在每个 row 内完成 16 个线程间的数据混洗。

接口声明

```
int __builtin_mxc_mov_shfl(int val, int mode, int rmask, int bmask, int bc);
```

参数

表 3.13 __builtin_mxc_mov_shfl 参数说明

名称	描述
dest	返回值
src	输入数据
mode	混洗模式。取值范围及对应操作如下： 0x000-0x0FF: Quad Permutation 0x101-0x10F: Row Shift Left 0x111-0x11F: Row Shift Right 0x121-0x12F: Row Rotate Right 0x140: Row Mirror 0x150-0x15f: Row Broadcast
RMSK	row mask。每个 bit 表示相应的 row 是否执行此次混洗。 if (!(rmask & 0x1)) row0 are disabled if (!(rmask & 0x2)) row1 are disabled if (!(rmask & 0x4)) row2 are disabled if (!(rmask & 0x8)) row3 are disabled

下页继续

表 3.13 – 续上页

名称	描述
BMSK	if (!(bmsk & 0x1)) lanes[0:3, 16:19, 32:35, 48:51] are disabled if (!(bmsk & 0x2)) lanes[4:7, 20:23, 36:39, 52:55] are disabled if (!(bmsk & 0x4)) lanes[8:11, 24:27, 40:43, 56:59] are disabled if (!(bmsk & 0x8)) lanes[12:15, 28:31, 44:47, 60:63] are disabled
BC	如果 BC 为 0, 则索引计算结果越界的线程将写 0 如果 BC 为 1, 则索引计算结果越界的线程值不变

算法

• Row Broadcast

将某个特定线程的值广播到同一个 row 内的所有其他线程。

```
for n in (1..64)
    dest.thread[n] = src.thread[(n&0x30)+ mode[3:0]]
```

• Quad Permutation

将一个 row 内的 16 个线程划分为 4 个连续的 4 线程区间, 在每一个区间内完成给定规则的混洗操作。

```
for n in (1..64)
    dest.thread[n] = src.thread[(n&0x3c)+mode[n%4*2+1:n%4*2]]
```

• Row Rotate Right

在一个 row 内的 16 个线程间进行循环右移。

```
for n in (1..64)
    If((n&0xf) >= mode[3:0]) {
        dest.thread[n] = src.thread[n-mode[3:0]];
    } else {
        dest.thread[n] = src.thread[n+16-mode[3:0]];
    }
```

• Row Shift Left

在一个 row 内的 16 个线程间进行左移。

```
for n in (1..64)
    If((n&0xf) < (16-mode[3:0])) {
        dest.thread[n] = src.thread[n+mode[3:0]];
    } else {
        if BC == 0:
            dest.thread[n] = 0
        else:
            do nothing
    }
```

• Row Shift Right

在一个 row 内的 16 个线程间进行右移。

```
for n in (1..64)
  If ((n&0xf) >= mode[3:0]) {
    dest.thread[n] = src.thread[n-mode[3:0]];
  } else {
    if BC == 0:
      dest.thread[n] = 0
    else:
      do nothing
  }
```

• Row Mirror

在一个 row 内的 16 个线程间进行镜像变换。

```
for n in (1..64)
  dest.thread[n] = src.thread[(n^0xf)].
```

代码示例

```
__device__ int test_mxc_mov_shfl(int val) {
  // Reverse every 16 threads.
  return __builtin_mxc_mov_shfl(val, 0x140, 0xf, 0xf, false);
}
```

3.4.2 __builtin_mxc_update_shfl

功能描述

功能与 __builtin_mxc_mov_shfl 类似，不同之处在于 __builtin_mxc_update_shfl 在线程索引计算结果越界时可以指定返回值。

接口声明

```
int __builtin_mxc_update_shfl(int old_val, int src, int mode, int rmask, int bmask,
    ↪ int bc);
```

参数

表 3.14 __builtin_mxc_update_shfl 参数说明

名称	描述
dest	返回值
old_val	线程索引计算结果越界且 BC 为 1 时的返回值
src	输入数据

下页继续

表 3.14 – 续上页

名称	描述
mode	混洗模式。取值范围及对应操作如下： 0x000-0x0FF: Quad Permutation 0x101-0x10F: Row Shift Left 0x111-0x11F: Row Shift Right 0x121-0x12F: Row Rotate Right 0x140: Row Mirror 0x150-0x15f: Row Broadcast
RMSK	row mask。每个 bit 表示相应的 row 是否执行此次混洗。 if (!(rmsk & 0x1)) row0 are disabled if (!(rmsk & 0x2)) row1 are disabled if (!(rmsk & 0x4)) row2 are disabled if (!(rmsk & 0x8)) row3 are disabled
BMSK	if (!(bmsk & 0x1)) lanes[0:3, 16:19, 32:35, 48:51] are disabled if (!(bmsk & 0x2)) lanes[4:7, 20:23, 36:39, 52:55] are disabled if (!(bmsk & 0x4)) lanes[8:11, 24:27, 40:43, 56:59] are disabled if (!(bmsk & 0x8)) lanes[12:15, 28:31, 44:47, 60:63] are disabled
BC	如果 BC 为 0，则索引计算结果越界的线程将写 0 如果 BC 为 1，则索引计算结果越界的线程将写 old_val

Algorithm

see __builtin_mxc_mov_shfl

代码示例

```
__device__ int test_mxc_mov_shfl(int val) {
    // Left shift every 16 threads, out of range return 100.
    return __builtin_mxc_update_shfl(100, val, 0x108, 0xf, 0xf, true);
}
```

3.4.3 __builtin_mxc_bsm_bpermute

功能描述

warp 内所有 64 个 thread 间的数据混洗。遍历返回值的所有线程，根据混洗规则参数选择输入数据特定线程的值写到返回值中。

接口声明

```
int __builtin_mxc_bsm_bpermute(int index, int data)
```

参数

表 3.15 __builtin_mxc_bsm_bpermute 参数说明

名称	描述
dest	返回值
index	混洗规则，用法见算法描述
data	输入数据

算法

```
for n in 0..63 //thread index
    dest[n] = data[floor(index[n] / 4) % 64]
```

代码示例

```
__device__ int test_mxc_bsm_bpermute(int index, int data) {
    // index[0:63] = {256, 254, ... 4, 0}
    // It will reverse the data of threads 0 to 63.
    return __builtin_mxc_bsm_bpermute(index, data);
}
```

3.4.4 __builtin_mxc_bsm_permute

功能描述

warp 内所有 64 个 thread 间的数据混洗。遍历输入数据的所有线程，根据混洗规则参数将输入数据写到返回值的特定线程中。

接口声明

```
int __builtin_mxc_bsm_permute(int index, int data)
```

参数

表 3.16 __builtin_mxc_bsm_permute 参数说明

名称	描述
dest	返回值
index	混洗规则，用法见算法描述
data	输入数据

算法

```
for n in 0..63 //thread index
    dest[floor(index[n] / 4) % 64] = data[n]
```

代码示例

```
__device__ int test_mxc_bsm_permute(int index, int data) {
    // index[0:63] = {256, 254, ... 4, 0}
    // It will reverse the data of threads 0 to 63.
    return __builtin_mxc_bsm_permute(index, data);
}
```

3.4.5 __builtin_mxc_byte_perm

功能描述

线程内两个 32bit 数据的字节混洗，返回一个新的 32bit 数据

接口声明

```
unsigned int __builtin_mxc_byte_perm(unsigned int s0, unsigned int s1, unsigned
↪perm);
```

参数

表 3.17 __builtin_mxc_byte_perm 参数说明

名称	描述
dest	dest[31:24] = byte_permute({s0,s1}, perm[31:24]) dest[23:16] = byte_permute({s0,s1}, perm[23:16]) dest[15:8] = byte_permute({s0,s1}, perm[15:8]) dest[7:0] = byte_permute({s0,s1}, perm[7:0])
s0	参与混洗的高 32bit 数据
s1	参与混洗的低 32bit 数据
perm	混洗规则，用法见算法描述

算法

```
uint8_t byte_permute(uint8_t in[8], uint8_t sel) {
    if(sel>=13) then return 0xff;
    elseif(sel==12) then return 0x00;
    elseif(sel==11) then return in[7][7] * 0xff;
    elseif(sel==10) then return in[5][7] * 0xff;
    elseif(sel==9) then return in[3][7] * 0xff;
    elseif(sel==8) then return in[1][7] * 0xff;
    else then return in[sel];
}

dest[31:24] = byte_permute({s0,s1}, perm[31:24]);
dest[23:16] = byte_permute({s0,s1}, perm[23:16]);
dest[15:8] = byte_permute({s0,s1}, perm[15:8]);
dest[7:0] = byte_permute({s0,s1}, perm[7:0]);
```

代码示例

```
__device__ int test_mxc_byte_reverse(unsigned int data) {  
    return __builtin_mxc_byte_perm(data, data, 0x04050607);  
}
```

3.5 浮点乘加函数

3.5.1 __builtin_mxc_pk_fma_f32

功能描述

单精度浮点向量 fused multiply add。

接口声明

```
v2f32 __builtin_mxc_pk_fma_f32(v2f32 a, v2f32 b, v2f32 c);
```

代码示例

```
using v2f32 = __NATIVE_VECTOR__(2, float);  
__global__ void test_mxc_pk_fma_f32(v2f32 *d, v2f32 *a, v2f32 *b, v2f32 *c) {  
    d[0] = __builtin_mxc_pk_fma_f32(a[0], b[0], c[0]);  
}
```

3.5.2 __builtin_mxc_pk_fma_bf16

架构支持限制

offload arch 需要大于等于 xcore1500。例如-offload-arch=xcore1500、-offload-arch=xcore1520 等。

功能描述

bf16 类型向量 fused multiply add。

接口声明

```
v2f16 __builtin_mxc_pk_fma_bf16(v2f16 a, v2f16 b, v2f16 c);
```

代码示例

```
using v2f16 = __NATIVE_VECTOR__(2, _Float16);  
__global__ void test_mxc_pk_fma_bf16(v2f16 *d, v2f16 *a, v2f16 *b, v2f16 *c) {  
    d[0] = __builtin_mxc_pk_fma_bf16(a[0], b[0], c[0]);  
}
```

3.5.3 __builtin_mxc_pk_fma_f16

架构支持限制

offload arch 需要大于等于 xcore1500。例如 -offload-arch=xcore1500、-offload-arch=xcore1520 等。

功能描述

fp16 浮点向量 fused multiply add。

接口声明

```
v2f16 __builtin_mxc_pk_fma_bf16(v2f16 a, v2f16 b, v2f16 c);
```

代码示例

```
using v2f16 = __NATIVE_VECTOR__(2, _Float16);
__global__ void test_mxc_pk_fma_f16(v2f16 *d, v2f16 *a, v2f16 *b, v2f16 *c) {
    d[0] = __builtin_mxc_pk_fma_f16(a[0], b[0], c[0]);
}
```

3.5.4 __builtin_mxc_fma_bf16

架构支持限制

offload arch 需要大于等于 xcore1500。例如 -offload-arch=xcore1500、-offload-arch=xcore1520 等。

功能描述

bf16 类型标量 fused multiply add。

接口声明

```
_Float16 __builtin_mxc_fma_bf16(_Float16 a, _Float16 b, _Float16 c);
```

代码示例

```
__global__ void test_mxc_fma_bf16(_Float16 *d, _Float16 *a, _Float16 *b, _Float16
↪ *c) {
    d[0] = __builtin_mxc_fma_bf16(a[0], b[0], c[0]);
}
```

3.6 矩阵乘加函数

3.6.1 __builtin_mxc_mma_16x16x16f16

功能描述

f16 类型矩阵乘加函数: $D = A * B + C$ 。

接口声明

```
v4f32 __builtin_mxc_mma_16x16x16f16(v4f16 a, v4f16 b, v4f32 c);
```

参数

表 3.18 __builtin_mxc_mma_16x16x16f16 参数说明

名称	描述
return_value	元素类型为 float 的 16x16 输出矩阵
a	元素类型为 f16 的 16x16 输入矩阵
b	元素类型为 f16 的 16x16 输入矩阵
c	元素类型为 float 的 16x16 输入矩阵

代码示例

```
using v4f16 = __NATIVE_VECTOR__(4, _Float16);
using v4f32 = __NATIVE_VECTOR__(4, float);
__device__ v4f32 test_builtin_mxc_mma_16x16x16f16(v4f16 a, v4f16 b, v4f32 c) {
    v4f32 d = __builtin_mxc_mma_16x16x16f16(a, b, c);
    return d;
}
```

3.6.2 __builtin_mxc_mma_16x16x4f32

功能描述

float 类型矩阵乘加函数: $D = A * B + C$.

接口声明

```
v4f32 __builtin_mxc_mma_16x16x4f32(float a, float b, v4f32 c);
```

参数

表 3.19 __builtin_mxc_mma_16x16x4f32 参数说明

名称	描述
return_value	元素类型为 float 的 16x16 输出矩阵
a	元素类型为 float 的 16x4 输入矩阵
b	元素类型为 float 的 4x16 输入矩阵
c	元素类型为 float 的 16x16 输入矩阵

代码示例

```
using v4f32 = __NATIVE_VECTOR__(4, float);
__device__ v4f32 test_builtin_mxc_mma_16x16x4f32(float a, float b, v4f32 c) {
    v4f32 d = __builtin_mxc_mma_16x16x4f32(a, b, c);
    return d;
}
```

3.6.3 __builtin_mxc_mma_16x16x16i8

架构支持限制

offload arch 需要小于 xcore1500。例如-offload-arch=xcore1000、-offload-arch=xcore1080 等。

功能描述

int8 类型矩阵乘加函数: $D = A * B + C$ 。

接口声明

```
v4i32 __builtin_mxc_mma_16x16x16i8(int a, int b, v4i32 c);
```

参数

表 3.20 __builtin_mxc_mma_16x16x16i8 参数说明

名称	描述
return_value	元素类型为 int32 的 16x16 输出矩阵
a	元素类型为 int8 的 16x16 输入矩阵
b	元素类型为 int8 的 16x16 输入矩阵
c	元素类型为 int32 的 16x16 输入矩阵

代码示例

```
using v4i32 = __NATIVE_VECTOR__(4, int);
__device__ v4i32 test_builtin_mxc_mma_16x16x16i8(int a, int b, v4i32 c) {
    v4i32 d = __builtin_mxc_mma_16x16x16i8(a, b, c);
    return d;
}
```

3.6.4 __builtin_mxc_mma_16x16x32i8

架构支持限制

offload arch 需要大于等于 xcore1500。例如 -offload-arch=xcore1500、-offload-arch=xcore1520 等。

功能描述

int8 type matrix mult-add : $D = A * B + C$.

接口声明

```
v4i32 __builtin_mxc_mma_16x16x32i8(v2i32 a, v2i32 b, v4i32 c);
```

参数

表 3.21 __builtin_mxc_mma_16x16x32i8 参数说明

名称	描述
return_value	元素类型为 int32 的 16x16 输出矩阵
a	元素类型为 int8 的 16x32 输入矩阵
b	元素类型为 int8 的 32x16 输入矩阵
c	元素类型为 int32 的 16x16 输入矩阵

代码示例

```
using v4i32 = __NATIVE_VECTOR__(4, int);
using v2i32 = __NATIVE_VECTOR__(2, int);
__device__ v4i32 test_builtin_mxc_mma_16x16x16i8(v2i32 a, v2i32 b, v4i32 c) {
    v4i32 d = __builtin_mxc_mma_16x16x32i8(a, b, c);
    return d;
}
```

3.6.5 __builtin_mxc_mma_16x16x16bf16

功能描述

bf16 类型矩阵乘加函数: $D = A * B + C$.

接口声明

```
v4f32 __builtin_mxc_mma_16x16x16bf16(v4f16 a, v4f16 b, v4f32 c);
```

参数

表 3.22 __builtin_mxc_mma_16x16x16bf16 参数说明

名称	描述
return_value	元素类型为 float 的 16x16 输出矩阵
a	元素类型为 bf16 的 16x16 输入矩阵
b	元素类型为 bf16 的 16x16 输入矩阵
c	元素类型为 float 的 16x16 输入矩阵

代码示例

```
using v4f16 = __NATIVE_VECTOR__(4, _Float16);
using v4f32 = __NATIVE_VECTOR__(4, float);
__device__ v4f32 test_builtin_mxc_mma_16x16x16bf16(v4f16 a, v4f16 b, v4f32 c) {
    v4f32 d = __builtin_mxc_mma_16x16x16bf16(a, b, c);
    return d;
}
```

3.6.6 __builtin_mxc_mma_16x16x4f64

功能描述

double 类型矩阵乘加函数: $D = A * B + C$.

接口声明

```
v4f64 __builtin_mxc_mma_16x16x4f64(double a, double b, v4f64 c);
```

参数

表 3.23 __builtin_mxc_mma_16x16x4f64 参数说明

名称	描述
return_value	元素类型为 double 的 16x16 输出矩阵
a	元素类型为 double 的 16x4 输入矩阵
b	元素类型为 double 的 4x16 输入矩阵
c	元素类型为 double 的 16x16 输入矩阵

代码示例

```
using v4f64 = __NATIVE_VECTOR__(4, double);
__device__ v4f64 test_builtin_mxc_mma_16x16x4f64(double A, double B, v4f64 C) {
    v4f64 D = __builtin_mxc_mma_16x16x4f64(A, B, C);
    return D;
}
```

3.6.7 __builtin_mxc_mma_16x16x8tf32

功能描述

tf32 类型矩阵乘加函数: $D = A * B + C$.

接口声明

```
v4f32 __builtin_mxc_mma_16x16x8tf32(v2f32 a, v2f32 b, v4f32 c);
```

参数

表 3.24 __builtin_mxc_mma_16x16x8tf32 参数说明

名称	描述
return_value	元素类型为 tf32 的 16x16 输出矩阵
a	元素类型为 tf32 的 16x8 输入矩阵
b	元素类型为 tf32 的 8x16 输入矩阵
c	元素类型为 tf32 的 16x16 输入矩阵

代码示例

```
using v4f32 = __NATIVE_VECTOR__(4, float);
using v2f32 = __NATIVE_VECTOR__(2, float);
__device__ v4f32 test_builtin_mxc_mma_16x16x8tf32(v2f32 A, v2f32 B, v4f32 C) {
    v4f32 D = __builtin_mxc_mma_16x16x8tf32(A, B, C);
    return D;
}
```

3.7 常用计算函数

3.7.1 __builtin_mxc_mad_wide_i32/u32

功能描述

整数乘加。其中乘法操作数宽度为 32bit，而累加操作数宽度为 64bit。

接口声明

```
int64_t __builtin_mxc_mad_wide_i32(int a, int b, int64_t c);
uint64_t __builtin_mxc_mad_wide_u32(unsigned int a, unsigned int b, uint64_t c);
```

参数

表 3.25 __builtin_mxc_mad_wide 参数说明

名称	描述
return_value	64bit 返回值
a	32bit 乘法操作数
b	32bit 乘法操作数
c	64bit 累加操作数

代码示例

```
__global__ void test_mxc_madi32(int a,int b,int64_t c,int64_t* d) {
    d[0]=__builtin_mxc_mad_wide_i32(a,b,c);
}

__global__ void test_mxc_madu32(unsigned int a,unsigned int b,uint64_t c,uint64_t*
↪d) {
    d[0]=__builtin_mxc_mad_wide_u32(a,b,c);
}
```

3.7.2 __builtin_mxc_pk_mul_f32

功能描述

单精度浮点向量乘法。d[0] = a[0] * b[0] d[1] = a[1] * b[1]

接口声明

```
v2f32 __builtin_mxc_pk_mul_f32(v2f32 a, v2f32 b);
```

代码示例

```
typedef __NATIVE_VECTOR__(2, float) v2f32;
__global__ void pkFmulKernel(v2f32* input, v2f32* output, size_t size) {
    int idx = threadIdx.x;
    output[idx] = __builtin_mxc_pk_mul_f32(input[idx], output[idx]);
}
```

3.7.3 __builtin_mxc_pk_add_f32

功能描述

单精度浮点向量加法。d[0] = a[0] + b[0] d[1] = a[1] + b[1]

接口声明

```
v2f32 __builtin_mxc_pk_add_f32(v2f32 a, v2f32 b);
```

代码示例

```
typedef __NATIVE_VECTOR__(2, float) v2f32;
__global__ void pkFmulKernel(v2f32* input, v2f32* output) {
    int idx = threadIdx.x;
    output[idx] = __builtin_mxc_pk_add_f32(input[idx], output[idx]);
}
```

3.7.4 __builtin_mxc_ubfe

功能描述无符号位域提取。

接口声明

```
unsigned int __builtin_mxc_ubfe(unsigned int data, unsigned int field_offset,
↪ unsigned int field_width);
```

参数

表 3.26 __builtin_mxc_ubfe 参数说明

名称	描述
return_value	返回值
data	输入数据
field_offset	要提取的位域的起始位置
field_width	要提取的位域的宽度

算法

```
return_value = (data >> field_offset[4:0]) & ((1 << field_width[4:0]) - 1)
```

代码示例

```
__global__ static void test_ubfe(unsigned int *res, unsigned int *src, unsigned
↪ int *offset, unsigned int *field) {
    unsigned int id = threadIdx.x;
    res[id] = __builtin_mxc_ubfe(src[id], offset[id], field[id]);
}
```

3.7.5 __builtin_mxc_sbfe

功能描述

有符号位域提取。

接口声明

```
int __builtin_mxc_sbfe(unsigned int data, unsigned int field_offset, unsigned int
↪field_width);
```

参数

表 3.27 __builtin_mxc_sbfe 参数说明

名称	描述
return_value	返回值
data	输入数据
field_offset	要提取的位域的起始位置
field_width	要提取的位域的宽度

算法

```
return_value = signext((data >> field_offset[4:0]) & ((1 << field_width[4:0]) -
↪1));
```

代码示例

```
__global__ static void test_sbfe(int *res, unsigned int *src, unsigned int *offset,
↪ unsigned int *field) {
    unsigned int id = threadIdx.x;
    res[id] = __builtin_mxc_sbfe(src[id], offset[id], field[id]);
}
```

3.7.6 __builtin_mxc_alignbit

功能描述

漏斗移位。

接口声明

```
unsigned int __builtin_mxc_alignbit(unsigned int a, unsigned int b, unsigned int
↪shift);
```

参数

表 3.28 __builtin_mxc_alignbit 参数说明

名称	描述
return_value	返回值
a	移位数据的高 32bit
b	移位数据的低 32bit
shift	移位位数

算法

```
return_value = ({a,b} >> shift[4:0]) & 0xffffffff;
```

代码示例

```
__global__ void test_alignbit_if_else(unsigned int *res) {
    if (__lane_id() & 0x1) {
        res[__lane_id()] = __builtin_mxc_alignbit(2, 1, 8);
    } else {
        res[__lane_id()] = __builtin_mxc_alignbit(2, 1, 4);
    }
}
```

3.7.7 __builtin_mxc_cos*

功能描述

float 和 fp16 类型的正弦函数。

接口声明

```
float __builtin_mxc_cosf(float);
_Float16 __builtin_mxc_cosh(_Float16);
```

3.7.8 __builtin_mxc_sin*

功能描述

float 和 fp16 类型的余弦函数。

接口声明

```
float __builtin_mxc_sinf(float);
_Float16 __builtin_mxc_sinh(_Float16);
```

3.7.9 __builtin_mxc_rcp*

功能描述

float 和 fp16 类型的倒数运算。

接口声明

```
float __builtin_mxc_rcpf(float);  
_Float16 __builtin_mxc_rcph(_Float16);
```

代码示例

```
__global__ void test_rcp(float * fres, float* fin, _Float16 *hres, _Float16 *hin) {  
    unsigned int id = threadIdx.x;  
    fres[id] = __builtin_mxc_rcpf(fin[id]);  
    hres[id] = __builtin_mxc_rcph(hin[id]);  
}
```

3.7.10 __builtin_mxc_sqrt*

功能描述

float 和 fp16 类型的平方根运算

接口声明

```
float __builtin_mxc_sqrtf(float src);  
_Float16 __builtin_mxc_sqrth(_Float16 src);
```

代码示例

```
__global__ void test_sqrt(float * fres, float* fin, _Float16 *hres, _Float16 *hin)  
↪ {  
    unsigned int id = threadIdx.x;  
    fres[id] = __builtin_mxc_sqrtf(fin[id]);  
    hres[id] = __builtin_mxc_sqrth(hin[id]);  
}
```

3.7.11 __builtin_mxc_rsq*

功能描述

float 和 fp16 类型的平方根倒数运算： $1.0/\sqrt{x}$ 。

接口声明

```
float __builtin_mxc_rsqf(float src);  
_Float16 __builtin_mxc_rsqh(_Float16 src);
```

代码示例

```
__device__ float test_mxc_rsqr(float a) {  
    return __builtin_mxc_rsqf(a);  
}
```

3.7.12 __builtin_mxc_log_clampf

功能描述

以 2 为底的对数运算。结果限制在 [0,1] 区间，当结果小于 0 时返回 0，当结果大于 1 时返回 1。

接口声明

```
float __builtin_mxc_log_clampf(float);
```

代码示例

```
__global__ void test_mxc_log_clampf(float& fres, float& f1) {  
    fres = __builtin_mxc_log_clampf(f1);  
}
```

3.8 其他

3.8.1 __builtin_mxc_get_time

功能描述

返回当前时刻的 64bit 时间戳。

接口声明

```
int64_t __builtin_mxc_get_time();
```

代码示例

```
__device__ int64_t test_mxc_get_time() {  
    return __builtin_mxc_get_time();  
}
```

3.8.2 __builtin_mxc_read_xmsk

功能描述

获取当前位置的线程活跃掩码。

接口声明

```
int64_t __builtin_mxc_read_xmsk();
```

代码示例

```
__device__ unsigned test_mxc_read_xmsk() {  
    return __builtin_mxc_read_xmsk();  
}
```

3.8.3 __builtin_mxc_readfirstlane

功能描述

获取输入数据的第一个活跃线程中的数据，如果没有活跃线程则返回线程 0 的数据。例如，当线程活跃掩码为 0xffffffff00000000 时，返回线程 32 中的值。

接口声明

```
int __builtin_mxc_readfirstlane(int data)
```

代码示例

```
__global__ void test_builtin_mxc_readfirstlane(int *output, int *input) {  
    int tid = blockDim.x * blockIdx.x + threadIdx.x;  
    int tmp;  
    // thread 32 ~ 63 is active.  
    if (tid > 31 && tid < 64) {  
        // return input[32].  
        tmp = __builtin_mxc_readfirstlane(input[tid]);  
    }  
    output[0] = tmp;  
}
```

3.8.4 __builtin_mxc_readlane

功能描述

获取输入数据的指定线程中的数据。

接口声明

```
int __builtin_mxc_readlane(int data, int thread)
```

参数

表 3.29 __builtin_mxc_readlane 参数说明

名字	描述
return_value	返回值
data	输入数据
thread	线程索引

代码示例

```
__device__ int test_mxc_readlane(int data, int thread) {  
    return __builtin_mxc_readlane(data, thread);  
}
```

3.8.5 __builtin_mxc_writelane

功能描述

向输入数据的指定线程中写入新值。

接口声明

```
int __builtin_mxc_writelane(int val, int lane, int old_data)
```

参数

表 3.30 __builtin_mxc_writelane 参数说明

名字	描述
val	要写入的新值
thread	线程索引
old_data	被修改的旧值

代码示例

```
__device__ int test_write_lane(int old_data) {  
    int ret_val = __builtin_mxc_writelane(10, 0, old_data);  
    return ret_val;  
}
```

3.8.6 __builtin_mxc_nop

功能描述

插入一定数量的空操作。

接口声明

```
int __builtin_mxc_nop(int n)
```

参数

表 3.31 __builtin_mxc_nop 参数说明

名字	描述
n	空操作数量。每个空操作等于 4 拍。

代码示例

```
__device__ void test_mxc_nop() {  
    __builtin_mxc_nop(32); // wait for 128 cycles  
    return;  
}
```